```
const express = require("express");
const fs = require("fs");
const path = require("path");
const mongoose = require("mongoose");
const Project = require("../models/Project");

const router = express.Router();
const DATA_DIR = path.join(__dirname, "../data");
const PROJECTS_FILE = path.join(DATA_DIR, "projects.json");

function ensureStorage() {
  if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });
  if (!fs.existsSync(PROJECTS_FILE)) fs.writeFileSync(PROJECTS_FILE, "[]",
"utf8");
}
function readLocalProjects() {
  ensureStorage();
  try { return JSON.parse(fs.readFileSync(PROJECTS_FILE, "utf8")); }
  catch { fs.writeFileSync(PROJECTS_FILE, "[]", "utf8"); return []; }
}
function writeLocalProjects(projects) {
  ensureStorage();
  fs.writeFileSync(PROJECTS_FILE, JSON.stringify(projects, null, 2), "utf8");
}
function mongoAvailable() { return mongoose.connection.readyState === 1; }

router.get("/", async (req, res) => {
  try {
    if (mongoAvailable()) return res.status(200).json(await Project.find().sort({
createdAt: -1 }));
    return res.status(200).json(readLocalProjects());
  } catch (error) {
    console.error("GET PROJECTS ERROR:", error);
    res.status(500).json({ success: false, message: "Error fetching projects" });
  }
});

router.post("/", async (req, res) => {
  try {
    const b = req.body || {};
    const projectName = typeof b.projectName === "string" ? b.projectName.trim()
: "";
    const location = typeof b.location === "string" ? b.location.trim() : "";
    const contractor = typeof b.contractor === "string" ? b.contractor.trim() :
"";
```

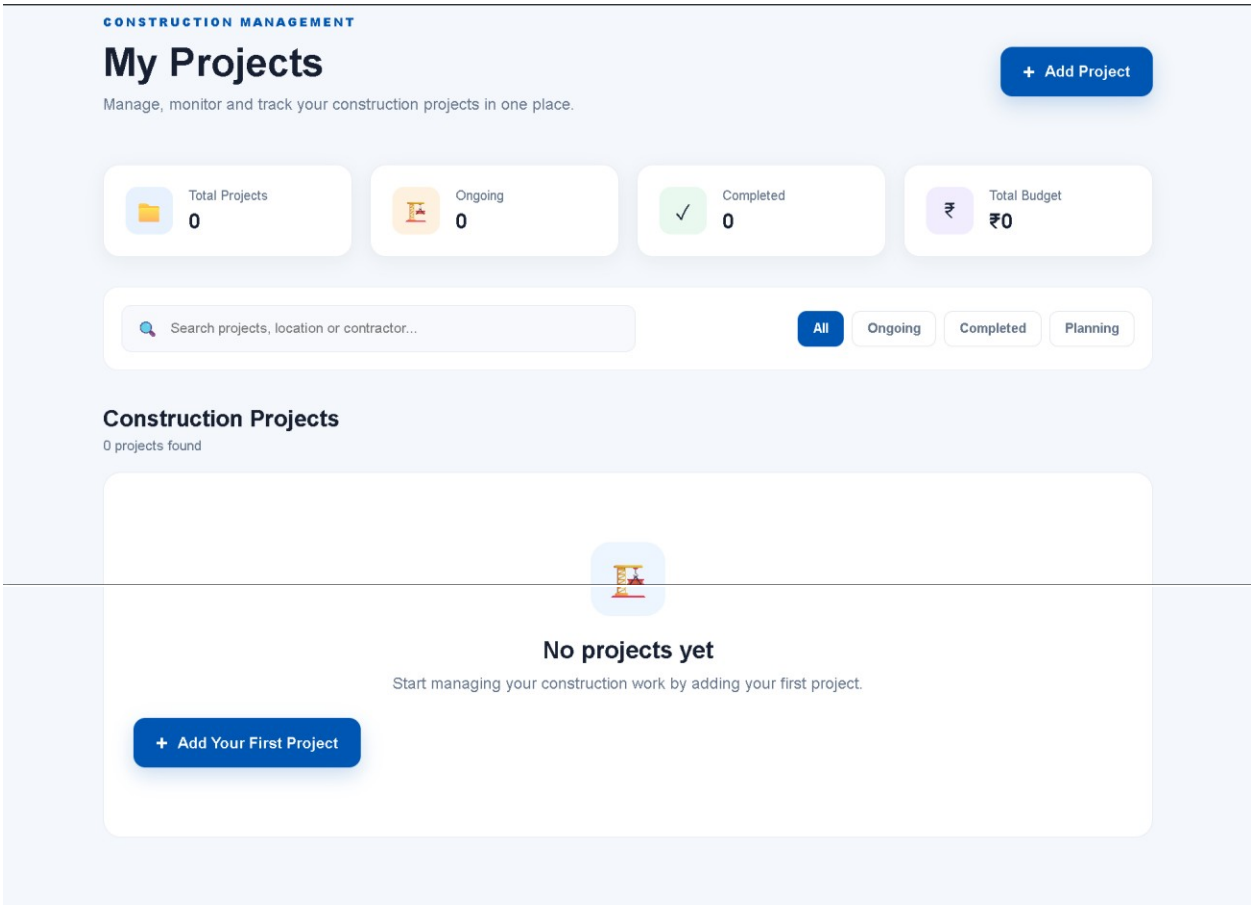
```
    const budget = Number(b.budget);
    if (!projectName || !location || !contractor || !Number.isFinite(budget) ||
budget <= 0) {
      return res.status(400).json({ success: false, message: "Project name,
location, contractor and valid budget are required" });
    }

    const data = {
      projectName,
      projectType: b.projectType || "House Construction",
      location,
      contractor,
      budget,
      budgetUsed: Math.max(0, Number(b.budgetUsed) || 0),
      progress: Math.min(100, Math.max(0, Number(b.progress) || 0)),
      status: b.status || "Ongoing",
      startDate: b.startDate || null,
      expectedCompletion: b.expectedCompletion || null,
      description: b.description || ""
    };

    if (mongoAvailable()) {
      const saved = await Project.create(data);
      return res.status(201).json({ success: true, message: "Project created
successfully", project: saved });
    }

    const project = { id: `local-${Date.now()}-
${Math.random().toString(36).slice(2,8)}`, ...data, createdAt: new
Date().toISOString() };
    const projects = readLocalProjects();
    projects.unshift(project);
    writeLocalProjects(projects);
    return res.status(201).json({ success: true, message: "Project created
successfully", project });
  } catch (error) {
    console.error("CREATE PROJECT ERROR:", error);
    res.status(500).json({ success: false, message: "Error creating project" });
  }
});

module.exports = router;
```


CONSTRUCTION MANAGEMENT
My Projects
Manage, monitor and track your construction projects in one place.
+ Add Project
Total Projects
0
Ongoing
0
Completed
0
Total Budget
₹0
Search projects, location or contractor...
All
Ongoing
Completed
Planning
Construction Projects
0 projects found
No projects yet
Start managing your construction work by adding your first project.
+ Add Your First Project